

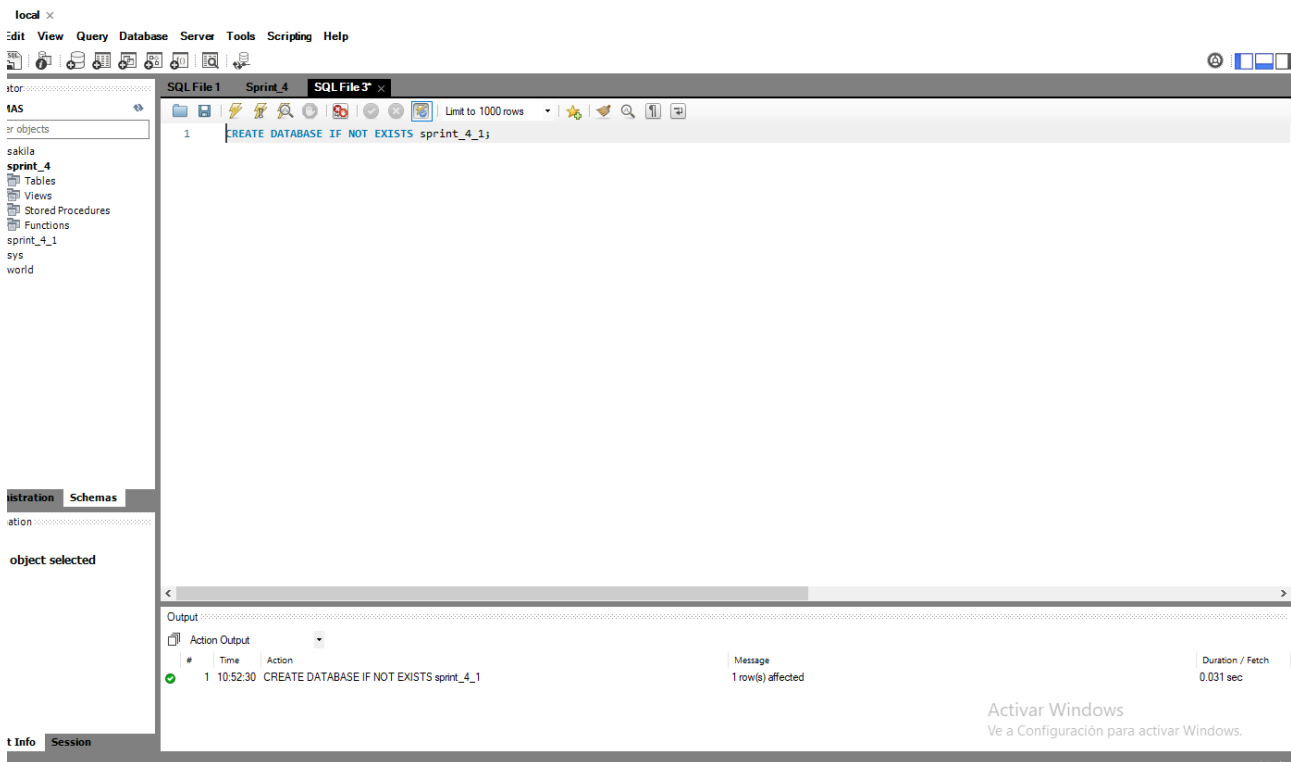
Sprint 4

Sergii Korotun

Nivel 1

Descarga los archivos CSV, estudiales y diseña una base de datos con un esquema de estrella que contenga, al menos 4 tablas de las que puedas realizar las siguientes consultas:

Creamos una base de datos



creamos la tabla “Transactions”

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'sprint_4_1' database schema with tables, views, stored procedures, and functions. The central pane shows the SQL script for creating the 'transaction' table. The script includes comments and the table definition with various data types and constraints. The bottom pane shows the execution output, indicating that the table was created successfully.

```
1 CREATE DATABASE IF NOT EXISTS sprint_4_1; -- creamos base de datos
2
3 -- creamos tablas --
4
5 CREATE TABLE IF NOT EXISTS transaction (
6
7     id VARCHAR (255) PRIMARY KEY ,
8     card_id VARCHAR (20) ,
9     business_id VARCHAR (255) ,
10    timestamp TIMESTAMP ,
11    amount DECIMAL (10,2) ,
12    declined BOOLEAN ,
13    product_ids VARCHAR (20) ,
14    user_id VARCHAR (255) ,
15    lat FLOAT ,
16    longitude FLOAT );
```

#	Time	Action	Message	Duration / Fetch
1	10:52:30	CREATE DATABASE IF NOT EXISTS sprint_4_1	1 row(s) affected	0.031 sec
2	10:57:49	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR (255) PRIMARY KEY , card_id VARCHAR...	0 row(s) affected	0.141 sec

Creamos la tabla “USERS”

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'sprint_4_1' database schema. The central pane shows the SQL script for creating the 'users' table. The script includes the table definition with various data types and constraints. The bottom pane shows the execution output, indicating that the table was created successfully.

```
17
18 CREATE TABLE IF NOT EXISTS users (
19     id VARCHAR (255) PRIMARY KEY ,
20     name VARCHAR(50) ,
21     surname VARCHAR(50) ,
22     phone VARCHAR(25) ,
23     email VARCHAR(100) ,
24     birth_day VARCHAR (20) ,
25     country VARCHAR(50) ,
26     city VARCHAR(50) ,
27     postal_code VARCHAR(20) ,
28     address VARCHAR(150)
29 );
```

#	Time	Action	Message	Duration / Fetch
1	10:52:30	CREATE DATABASE IF NOT EXISTS sprint_4_1	1 row(s) affected	0.031 sec
2	10:57:49	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR (255) PRIMARY KEY , card_id VARCHAR...	0 row(s) affected	0.141 sec
3	10:59:59	CREATE TABLE IF NOT EXISTS users (id VARCHAR (255) PRIMARY KEY , name VARCHAR(50) , surname VARCHAR(50) , phone VARCHAR(25) , email VARCHAR(100) , birth_day VARCHAR (20) , country VARCHAR(50) , city VARCHAR(50) , postal_code VARCHAR(20) , address VARCHAR(150));	0 row(s) affected	0.093 sec

creamos tabla Credit_cards

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the database structure, including the 'sprint_4' database and its tables. The right pane shows the SQL script for creating the 'credit_cards' table. The script includes the following SQL code:

```
19 id VARCHAR (255) PRIMARY KEY ,
20 name VARCHAR(50) ,
21 surname VARCHAR(50) ,
22 phone VARCHAR(25),
23 email VARCHAR(100) ,
24 birth_day VARCHAR (20),
25 country VARCHAR(50),
26 city VARCHAR(50),
27 postal_code VARCHAR(20),
28 address VARCHAR(150)
29 );
30
31 CREATE TABLE IF NOT EXISTS credit_cards (
32 id VARCHAR (255) PRIMARY KEY ,
33 user_id VARCHAR(50) ,
34 iban VARCHAR(50) ,
35 pan VARCHAR(25),
36 pin VARCHAR(100) ,
37 cvv VARCHAR (20),
38 track1 VARCHAR(50),
39 track2 VARCHAR(50),
40 expiring_date VARCHAR(20)
41 );
```

The bottom pane shows the 'Output' window with the following table:

#	Time	Action	Message	Duration / Fetch
1	10:52:30	CREATE DATABASE IF NOT EXISTS sprint_4_1	1 row(s) affected	0.031 sec
2	10:57:49	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR (255) PRIMARY KEY , card_id VARCHAR...	0 row(s) affected	0.141 sec
3	10:59:59	CREATE TABLE IF NOT EXISTS users (id VARCHAR (255) PRIMARY KEY , name VARCHAR(5...	0 row(s) affected	0.093 sec
4	11:01:11	CREATE TABLE IF NOT EXISTS credit_cards (id VARCHAR (255) PRIMARY KEY , user_id VA...	0 row(s) affected	0.093 sec

creamos tabla Companies

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the database structure, including the 'sprint_4' database and its tables. The right pane shows the SQL script for creating the 'companies' table. The script includes the following SQL code:

```
28 address VARCHAR(150)
29 );
30
31 CREATE TABLE IF NOT EXISTS credit_cards (
32 id VARCHAR (255) PRIMARY KEY ,
33 user_id VARCHAR(50) ,
34 iban VARCHAR(50) ,
35 pan VARCHAR(25),
36 pin VARCHAR(100) ,
37 cvv VARCHAR (20),
38 track1 VARCHAR(50),
39 track2 VARCHAR(50),
40 expiring_date VARCHAR(20)
41 );
42
43 CREATE TABLE IF NOT EXISTS companies (
44 company_id VARCHAR (255) PRIMARY KEY ,
45 company_name VARCHAR(50) ,
46 phone VARCHAR(50) ,
47 email VARCHAR(25),
48 country VARCHAR(100) ,
49 website VARCHAR (20)
50 );
```

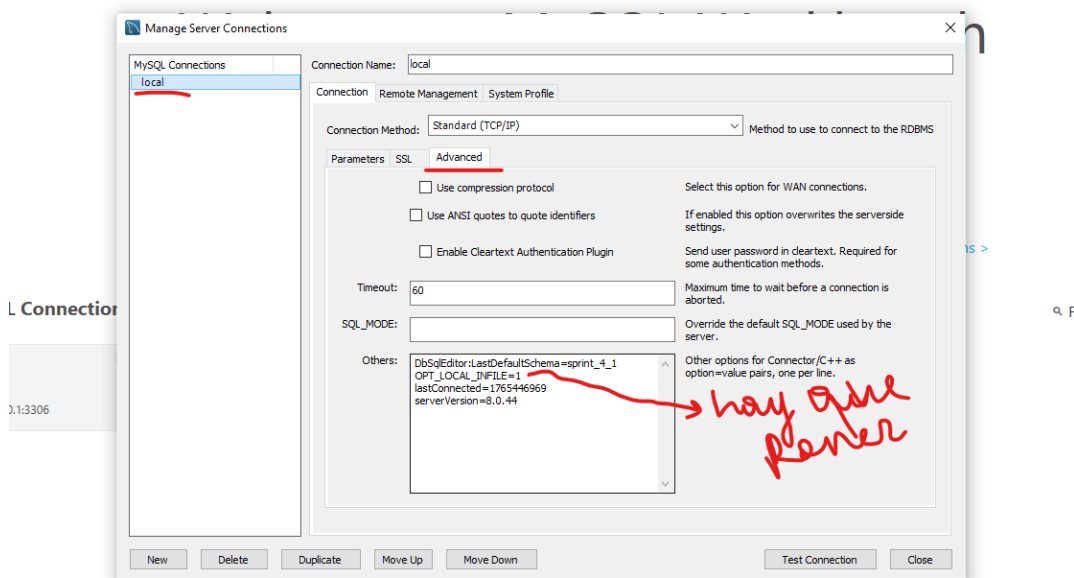
The bottom pane shows the 'Output' window with the following table:

#	Time	Action	Message	Duration / Fetch
2	10:57:49	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR (255) PRIMARY KEY , card_id VARCHAR...	0 row(s) affected	0.141 sec
3	10:59:59	CREATE TABLE IF NOT EXISTS users (id VARCHAR (255) PRIMARY KEY , name VARCHAR...	0 row(s) affected	0.093 sec
4	11:01:11	CREATE TABLE IF NOT EXISTS credit_cards (id VARCHAR (255) PRIMARY KEY , user_id VA...	0 row(s) affected	0.093 sec
5	11:02:40	CREATE TABLE IF NOT EXISTS companies (company_id VARCHAR (255) PRIMARY KEY , c...	0 row(s) affected	0.079 sec

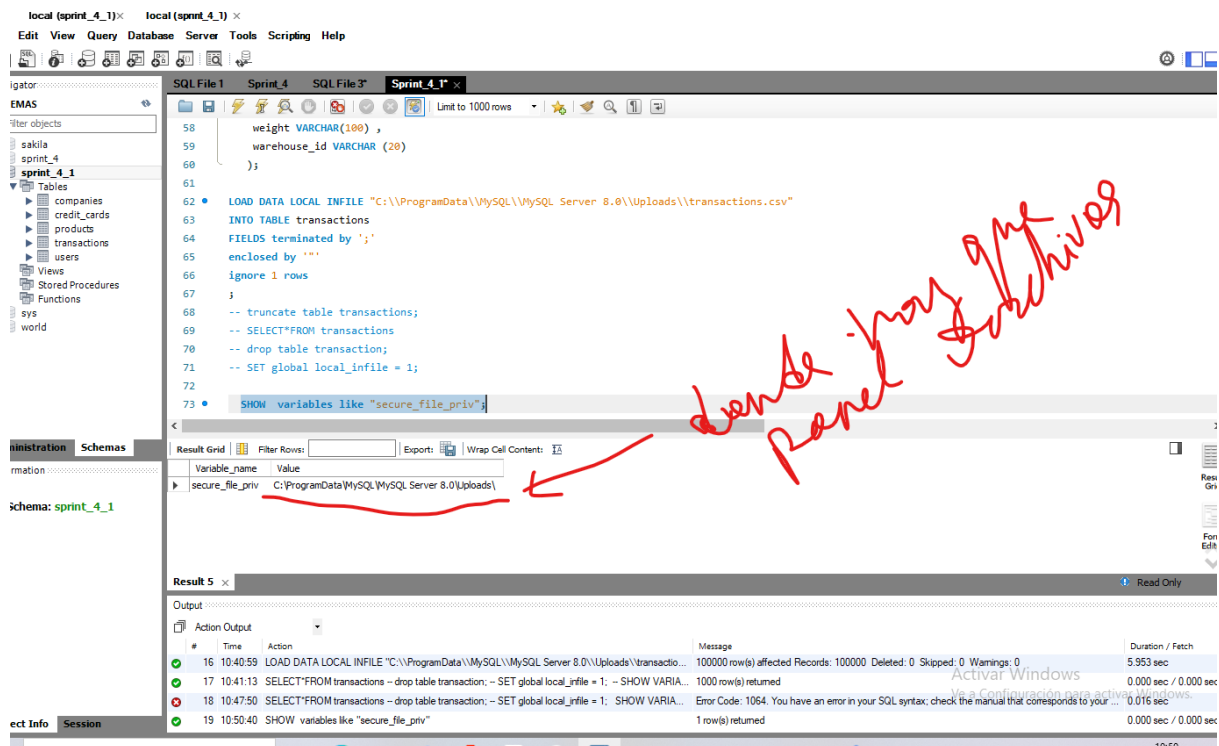
Pasamos los datos a las tablas para cargar_archivos

```
-- drop table transaction;  
• SET global local_infile = 1;  
• SHOW variables like "secure_file_priv";
```

← permite cargar datos



encontramos la carpeta donde se puede cargar los archivos



añadimos los datos a las tablas transacciones

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following code:

```
57 colour VARCHAR(25),
58 weight VARCHAR(100),
59 warehouse_id VARCHAR (20)
60 );
61
62 • LOAD DATA LOCAL INFILE "C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads\\transactions.csv"
63 INTO TABLE transactions
64 FIELDS terminated by ',' ← importante
65 enclosed by ''
66 ignore 1 rows
67 ;
68 -- truncate table transactions;
69 • SELECT*FROM transactions
70 -- drop table transaction;
71 -- SET global local_infile = 1;
72 -- SHOW VARIABLES LIKE 'secure_file_priv';
```

The SQL editor is set to "Limit to 1000 rows". The Results window shows the data imported into the 'transactions' table:

id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat	longitude
0004349-2949-4949-A58AE38B19D0	CcS-9294	b-2458	2024-08-28 07:16:46	395.43	0	16, 26, 97, 87	4713	46.1999	1.43554
000447FE-B650-40CF-85DE-C7ED0EE1CAAD	CcS-5019	b-2370	2016-12-21 20:07:18	155.63	0	66, 69, 87	438	41.5972	12.2218
00045D6B-ED2E-4F2F-8186-CE074D875D0D	CcS-6699	b-2390	2020-07-14 15:37:45	326.01	0	30, 11, 16, 81	2118	29.7573	-95.3796
000481C3-1C26-4FEF-83AD-4CD0E0048BD	CcS-6696	b-2230	2017-09-04 19:44:53	161.60	0	72	2115	53.5489	-113.503
00051AA4-9CBE-4268-B070-C38062A183E2	CcS-7606	b-2266	2017-01-05 18:19:25	148.91	0	18	3025	52.2084	5.69081
0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	CcU-3358	b-2598	2023-09-23 04:51:43	294.59	0	35, 33, 19	215	53.5535	-113.499

The Output window shows the execution results:

#	Time	Action	Message	Duration / Fetch
14	10:40:44	truncate table transactions	0 row(s) affected	0.188 sec
15	10:40:50	SELECT*FROM transactions -- drop table transaction; -- SET global local_infile = 1; -- SHOW VARIA...	1000 row(s) returned	0.000 sec / 0.000 sec
16	10:40:59	LOAD DATA LOCAL INFILE "C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads\\transac...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0	5.953 sec
17	10:41:13	SELECT*FROM transactions -- drop table transaction; -- SET global local_infile = 1; -- SHOW VARIA...	1000 row(s) returned	0.000 sec / 0.000 sec

añadimos los datos a la tabla “Users” pasamos datos american y european a la tabla unica

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following code:

```
50 country VARCHAR(100),
51 website VARCHAR (20)
52 );
53 • CREATE TABLE IF NOT EXISTS products (
54 id VARCHAR (255) PRIMARY KEY ,
55 product_name VARCHAR(50),
56 price VARCHAR(50),
57 colour VARCHAR(25),
58 weight VARCHAR(100),
59 warehouse_id VARCHAR (20)
60 );
61
62 • LOAD DATA LOCAL INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/american_users.csv'
63 INTO TABLE users
64 FIELDS terminated by ','
65 enclosed by ''
66 ignore 1 rows
67 ;
68
69 • select*from users
70
71 -- SET GLOBAL local_infile = 1;
72 -- SHOW VARIABLES LIKE 'secure_file_priv';
73 -- SHOW GLOBAL VARIABLES LIKE 'local_infile';
```

The SQL editor is set to "Limit to 1000 rows". The Results window shows the data imported into the 'users' table:

#	Time	Action	Message	Duration / Fetch
1	11:05:20	select*from transactions -- SET GLOBAL local_infile = 1; -- SHOW VARIABLES LIKE 'secure_file_priv'...	1000 row(s) returned	0.000 sec / 0.016 s
2	11:05:51	select*from transactions -- SET GLOBAL local_infile = 1; -- SHOW VARIABLES LIKE 'secure_file_priv'...	1000 row(s) returned	0.000 sec / 0.015 s
3	11:06:38	LOAD DATA LOCAL INFILE "C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads\\american_users.c...	Error Code: 2. File "C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads\\american_users.csv" not found (O...	0.015 sec
4	11:09:44	LOAD DATA LOCAL INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/american_users.c...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	0.171 sec

revisamos si todo hemos hecho bien

Query 1 Sprint_4 Sprint_4.1

```
67 ;
68
69 • LOAD DATA LOCAL INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/american_users.csv'
70 INTO TABLE users
71 FIELDS terminated by ','
72 enclosed by ''
73 ignore 1 rows
74 ;
75 • LOAD DATA LOCAL INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/european_users.csv'
76 INTO TABLE users
77 FIELDS terminated by ','
78 enclosed by ''
79 ignore 1 rows
80 ;
81
82 • select country from users
```

Result Grid

country
United States
Netherlands
Poland
France
Canada
United Kingdom

users 4

Output

#	Time	Action	Message	Duration / Fetch
5	11:12:57	select from users -- SET GLOBAL local_infile = 1; -- SHOW VARIABLES LIKE 'secure_file_priv'; -- S...	1000 row(s) returned	0.000 sec / 0.000 sec
6	11:20:21	LOAD DATA LOCAL INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/european_user...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0	0.391 sec
7	11:22:50	select coutry from users -- SET GLOBAL local_infile = 1; -- SHOW VARIABLES LIKE 'secure_file_pri...	Error Code: 1054. Unknown column 'coutry' in field list	0.000 sec
8	11:23:14	select country from users -- SET GLOBAL local_infile = 1; -- SHOW VARIABLES LIKE 'secure_file_p...	1000 row(s) returned	0.015 sec / 0.000 sec

añadimos los datos a la tabla credit_cards

Query 1 Sprint_4 Sprint_4.1

```
75 • LOAD DATA LOCAL INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/european_users.csv'
76 INTO TABLE users
77 FIELDS terminated by ','
78 enclosed by ''
79 ignore 1 rows
80 ;
81
82 • LOAD DATA LOCAL INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/credit_cards.csv'
83 INTO TABLE credit_cards
84 FIELDS terminated by ','
85 enclosed by ''
86 ignore 1 rows
87 ;
88
89 • select*from credit_cards
90
```

Result Grid

id	user_id	iban	pan	pin	cvv	track1	track2	expiring_date
CcS-4857	276	Xx4857591835292505850771	2314242385113924	1819	467	%82314242385113924~LWCBUDLWCBUD~22...	%82314242385113924~24101015183631647	09/27/25
CcS-4858	277	Xx83581768137002436094025	6582720299715533	3964	817	%68582720299715533~TIQWMTIQQWMT~2404...	%68582720299715533~2411101045462722	12/28/28
CcS-4859	278	Xx7826930491423553609370	8861684536289642	4983	277	%68861684536289642~COFBDGCOFBDG~280...	%68861684536289642~25021017616653717	11/26/26
CcS-4860	279	Xx5559590368835304645299	2481155515498459	6876	661	%82481155515498459~TIUJTUIJTUI~3104000000000000	%498459~26021015144143957	07/27/27
CcS-4861	280	Xx2035182877195191627307	1308930301149557	5710	398	%82481155515498459~TIUJTUIJTUI~3104000000000000	%149557~28051017513050287	04/25/26
CcS-4862	281	Xx4774721462463645409758	6715617009807829	4042	174	%86715617009807829~LDMVTDLDMVTD~33...	%86715617009807829~22101017023704287	11/27/26

credit_cards 5

Output

#	Time	Action	Message	Duration / Fetch
7	11:22:50	select coutry from users -- SET GLOBAL local_infile = 1; -- SHOW VARIABLES LIKE 'secure_file_priv'; -- SHOW GLOBAL VARIA...	Error Code: 1054. Unknown column 'coutry' in field list	0.000 sec
8	11:23:14	select country from users -- SET GLOBAL local_infile = 1; -- SHOW VARIABLES LIKE 'secure_file_priv'; -- SHOW GLOBAL VARIA...	1000 row(s) returned	0.015 sec / 0.000 sec
9	11:27:02	LOAD DATA LOCAL INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/credit_cards.csv' INTO TABLE credit_cards ...		0.812 sec
10	11:27:43	select from credit_cards -- SET GLOBAL local_infile = 1; -- SHOW VARIABLES LIKE 'secure_file_priv'; -- SHOW GLOBAL VARIA...	1000 row(s) returned	0.000 sec / 0.015 sec

revisamos si todo bien

```
84  FIELDS terminated by ','
85  enclosed by '''
86  ignore 1 rows
87  ;
88
89  • LOAD DATA LOCAL INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/companies.csv'
90  INTO TABLE companies
91  FIELDS terminated by ','
92  enclosed by '''
93  ignore 1 rows
94  ;
95
96  • LOAD DATA LOCAL INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/products.csv'
97  INTO TABLE products
98  FIELDS terminated by ','
99  enclosed by '''
100 ignore 1 rows
101 ;
102
103 -- select*from companies
104 • drop table products
105 -- SET GLOBAL local_infile = 1;
106 -- SHOW VARIABLES LIKE 'secure_file_priv';
107 -- SHOW GLOBAL VARIABLES LIKE 'local_infile';
```

Output

#	Time	Action	Message	Duration / Fetch
16	11:40:24	CREATE TABLE IF NOT EXISTS companies (company_id VARCHAR (255) PRIMARY KEY , company_name VARCHAR(...	0 row(s) affected	0.093 sec
17	11:40:35	LOAD DATA LOCAL INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/companies.csv' INTO TABLE companies FI...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.047 sec
18	11:41:43	LOAD DATA LOCAL INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/products.csv' INTO TABLE products FIELD...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.062 sec
19	11:42:12	drop table products -- SET GLOBAL local_infile = 1; -- SHOW VARIABLES LIKE 'secure_file_priv'; -- SHOW GLOBAL VARIABLE...	0 row(s) affected	0.110 sec

sigamos con mismo proceso y añadimos los datos a la tabla Companias

```
41  expiring_date VARCHAR(20)
42  );
43  • CREATE TABLE IF NOT EXISTS companies (
44  company_id VARCHAR (255) PRIMARY KEY ,
45  company_name VARCHAR(50) ,
46  phone VARCHAR(50) ,
47  email VARCHAR(25),
48  country VARCHAR(100) ,
49  website VARCHAR (20)
50  );
51
52
53
54
55
56
57
58
59
60
61
```

Output

#	Time	Action	Message	Duration / Fetch
3	11:05:51	drop table companies	0 row(s) affected	0.156 sec
4	11:06:47	CREATE TABLE IF NOT EXISTS companies (company_id VARCHAR (255) PRIMARY KEY , c...	0 row(s) affected	0.110 sec

añadimos los datos a la tabla Companies

The screenshot displays a MySQL command-line interface with the following SQL commands:

```
51
52 -- añadimos los datos a la tabla COMPANIES
53 • LOAD DATA LOCAL INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/companies.csv'
54 INTO TABLE companies
55 FIELDS terminated by ','
56 enclosed by ''
57 ignore 1 rows
58 ;
59
60 -- TRUNCATE TABLE companies;
61
62
63
64
65
66
67
68
69
```

The Output window shows the following messages:

#	Time	Action	Message	Duration / Fetch
10	11:14:55	select from companies	CREATE TABLE IF NOT EXISTS products (id VARCHAR...	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your ... 0.000 sec
11	11:16:38	LOAD DATA LOCAL INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/companies.csv' I...	0 row(s) affected, 236 warning(s): 1265 Data truncated for column 'email' at row 1 1265 Data truncate...	0.016 sec

The Windows taskbar at the bottom shows the system clock as 11:17 on 16/12/2025, with a temperature of 10°C and the word "Despejado" (Clear).

Ahora conectamos las tablas (FK)
Como vamos conectarlas

creamos FK
transaction con credit_cards

The screenshot displays a SQL IDE interface. The top pane shows SQL code for creating foreign key constraints. A red bracket highlights the first two lines of the code block:

```
104 -- creamos relaciones (FK) --  
105 ALTER TABLE transactions  
106 ADD CONSTRAINT card_id  
107 FOREIGN KEY (card_id) REFERENCES credit_cards(id);
```

The bottom pane shows the 'Output' window with the following table:

#	Time	Action	Message	Duration / Fetch
19	11:42:12	drop table products -- SET GLOBAL local_infile = 1; -- SHOW VARIABLES LIKE 'secure_file_priv'; -- SHOW GLOBAL VARIABLES ...	0 row(s) affected	0.110 sec
20	12:00:13	ALTER TABLE transaction ADD CONSTRAINT card_id FOREIGN KEY (card_id) REFERENCES card(id)	Error Code: 1146. Table 'sprint_4_1.transaction' doesn't exist	0.000 sec
21	12:00:59	ALTER TABLE transactions ADD CONSTRAINT card_id FOREIGN KEY (card_id) REFERENCES card(id)	Error Code: 1824. Failed to open the referenced table 'card'	0.000 sec
22	12:02:42	ALTER TABLE transactions ADD CONSTRAINT card_id FOREIGN KEY (card_id) REFERENCES credit_cards(id)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	3.796 sec

Below the output window, a diagram shows the database schema with four tables: **company**, **users**, **transactions**, and **credit_cards**. Red arrows indicate the foreign key relationships:

- From **transactions** (card_id) to **credit_cards** (id)
- From **transactions** (user_id) to **users** (id)
- From **transactions** (business_id) to **company** (company_id)

creamos FK

transaction con users

```
108
109 • ALTER TABLE transactions
110 ADD CONSTRAINT fk_user_id
111 FOREIGN KEY (user_id) REFERENCES users(id);
112
113 • ALTER TABLE transactions
114 ADD CONSTRAINT fk_card_id
115 FOREIGN KEY (card_id) REFERENCES credit_cards(id);
116
117 • ALTER TABLE transactions
118 ADD CONSTRAINT fk_business_id
119 FOREIGN KEY (business_id) REFERENCES companies(company_id);
120
121 • ALTER TABLE transactions
122 ADD CONSTRAINT fk_product_ids
123 FOREIGN KEY (product_ids) REFERENCES products(id);
124
```

→ add constraint with users

Output

#	Time	Action	Message	Duration / Fetch
20	12:00:13	ALTER TABLE transaction ADD CONSTRAINT card_id FOREIGN KEY (card_id) REFERENCES card(id)	Error Code: 1146. Table 'sprint_4_1.transaction' doesn't exist	0.000 sec
21	12:00:59	ALTER TABLE transactions ADD CONSTRAINT card_id FOREIGN KEY (card_id) REFERENCES card(id)	Error Code: 1824. Failed to open the referenced table 'card'	0.000 sec
22	12:02:42	ALTER TABLE transactions ADD CONSTRAINT card_id FOREIGN KEY (card_id) REFERENCES credit_cards(id)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	3.796 sec
23	12:05:50	ALTER TABLE transactions ADD CONSTRAINT fk_user_id FOREIGN KEY (user_id) REFERENCES users(id)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	4.625 sec

creamos FK transaction con business

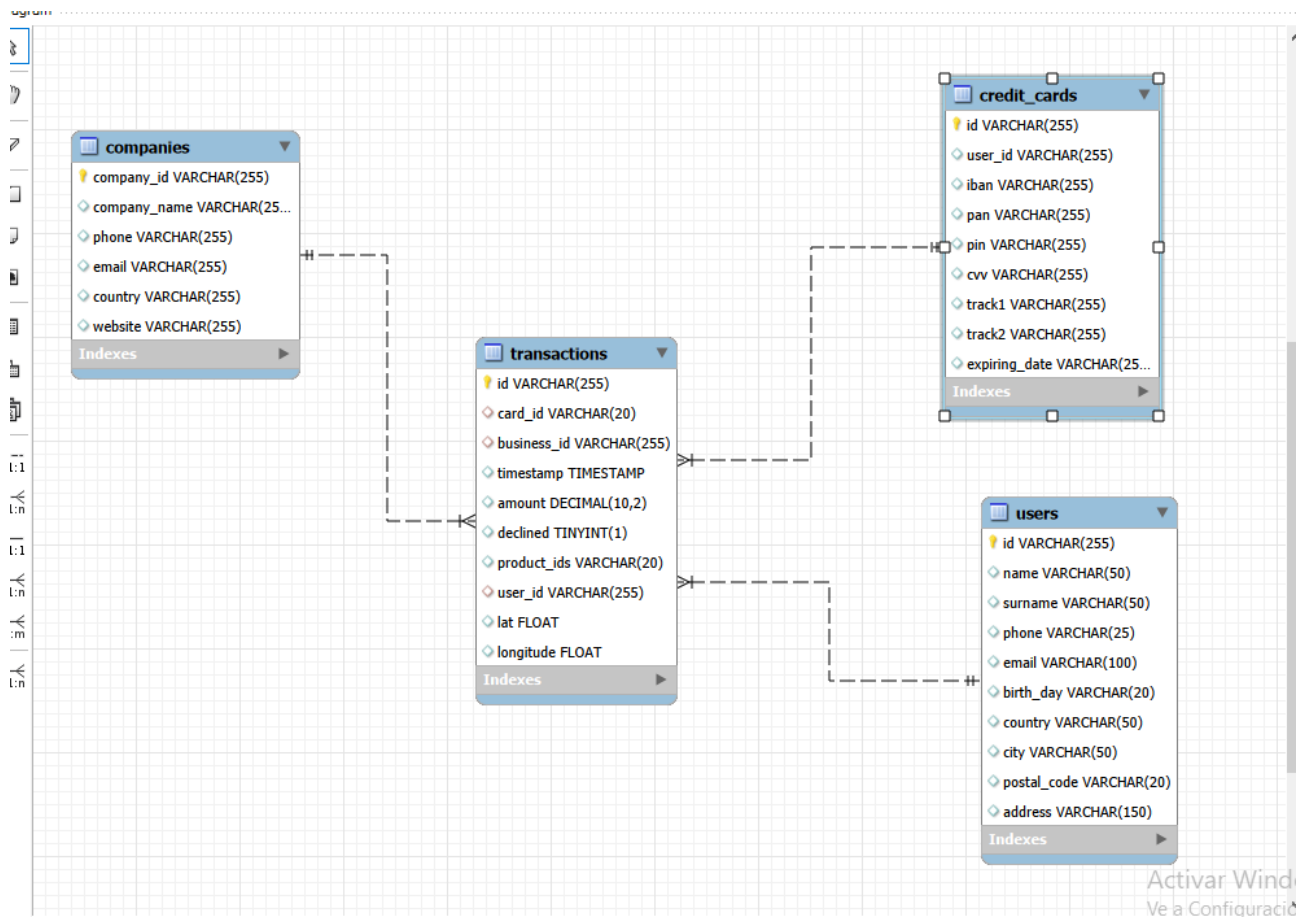
```
116
117 • ALTER TABLE transactions
118 ADD CONSTRAINT fk_business_id
119 FOREIGN KEY (business_id) REFERENCES companies(company_id);
120
121 • ALTER TABLE transactions
122 ADD CONSTRAINT fk_product_ids
123 FOREIGN KEY (product_ids) REFERENCES products(id);
124
125 -- select*from companies
126 -- drop table products
127 -- SET GLOBAL local_infile = 1;
128 -- SHOW VARIABLES LIKE 'secure_file_priv';
129 -- SHOW GLOBAL VARIABLES LIKE 'local_infile';
```

→ add constraint with business

Output

#	Time	Action	Message	Duration / Fetch
22	12:02:42	ALTER TABLE transactions ADD CONSTRAINT card_id FOREIGN KEY (card_id) REFERENCES credit_cards(id)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	3.796 sec
23	12:05:50	ALTER TABLE transactions ADD CONSTRAINT fk_user_id FOREIGN KEY (user_id) REFERENCES users(id)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	4.625 sec
24	12:15:22	ALTER TABLE transactions ADD CONSTRAINT fk_card_id FOREIGN KEY (card_id) REFERENCES credit_cards(id)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	6.375 sec
25	12:26:50	ALTER TABLE transactions ADD CONSTRAINT fk_business_id FOREIGN KEY (business_id) REFERENCES companies(company_id)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	5.359 sec

diagram



Ejercicio 1

Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.

```
122 -- doble conection por mi culpa, y deberia que borrar una de ellas.
123 • ALTER TABLE transactions
124 DROP FOREIGN KEY fk_card_id;
125
126 -- NIVEL-1-1-
127
128 • SELECT u.id, u.name, u.surname
129 FROM users u
130 WHERE u.id IN (
131     SELECT t.user_id
132     FROM transactions t
133     GROUP BY t.user_id
134     HAVING COUNT(t.id) > 80
135 )
136 limit 0,100;
```

Result Grid

id	name	surname
185	Molly	Gilliam
289	Dxwgi	Hwcru
318	Bnyr	Astuw
454	Sfzroh	Xgyfrdxs

users 6

Output

#	Time	Action	Message	Duration / Fetch
30	12:39:49	ALTER TABLE transactions DROP FOREIGN KEY fk_card_id	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.078 sec
31	12:40:15	ALTER TABLE transactions ADD CONSTRAINT fk_card_id FOREIGN KEY (card_id) REFERENCES credit_cards(id)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	5.563 sec
32	12:41:49	ALTER TABLE transactions DROP FOREIGN KEY fk_card_id	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.063 sec
33	12:47:25	SELECT u.id, u.name, u.surname FROM users u WHERE u.id IN (SELECT t.user_id FROM transactions t GROUP BY t.us...	4 row(s) returned	0.157 sec / 0.000

Ejercicio 2

Muestra la media de amount por IBAN de las tarjetas de crédito en la compañía Donec Ltd., utiliza por lo menos 2 tablas.

```
132 FROM transactions t
133 GROUP BY t.user_id
134 HAVING COUNT(t.id) > 80
135 )
136 limit 0,100;
137
138 -- NIVEL-1-2--
139
140 • SELECT cc.iban,
141     AVG(t.amount) AS media_amount
142 FROM transactions t
143 JOIN credit_cards cc ON t.card_id = cc.id
144 JOIN companies c ON t.business_id = c.company_id
145 WHERE c.company_name = 'Donec Ltd'
146 GROUP BY cc.iban;
```

Result Grid

iban	media_amount
XX911406401125586307586805	356.246667
SK9446370242474562577506	142.960000
XX776752917845952975555640	257.370000
XX413827362289719304908990	139.590000
XX347787246070769610780308	240.410000
XX688768436543090894854602	188.580000

Result 7

Output

#	Time	Action	Message	Duration / Fetch
31	12:40:15	ALTER TABLE transactions ADD CONSTRAINT fk_card_id FOREIGN KEY (card_id) REFERENCES credit_cards(id)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	5.563 sec
32	12:41:49	ALTER TABLE transactions DROP FOREIGN KEY fk_card_id	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.063 sec
33	12:47:25	SELECT u.id, u.name, u.surname FROM users u WHERE u.id IN (SELECT t.user_id FROM transactions t GROUP BY t.us...	4 row(s) returned	0.157 sec / 0.000 sec
34	12:48:31	SELECT cc.iban, AVG(t.amount) AS media_amount FROM transactions t JOIN credit_cards cc ON t.card_id = cc.id JOIN com...	371 row(s) returned	0.032 sec / 0.000 sec

Nivel 2

Crea una nueva tabla que refleje el estado de las tarjetas de crédito basado en si las tres últimas transacciones han sido declinadas entonces es inactivo, si por lo menos una no es rechazada entonces es activo. Partiendo de esta tabla responde:

Ejercicio 1

¿Cuántas tarjetas están activas?

Creamos una tabla status credit_card

```
148 -- NIVEL-2-- (creamos una tabla que se debuelva valor si la tarjeta esta activa o no )
149 • create table if not exists status_credit_cards (
150   credit_card_id varchar(255) primary key,
151   status_card enum ('active','inactive'),
152   FOREIGN KEY (credit_card_id) REFERENCES credit_cards(id)
153 );
154
155
156
157
158
159 -- select*from companies
160 -- drop table products
161 -- SET GLOBAL local_infile = 1;
162 -- SHOW VARIABLES LIKE 'secure_file_priv';
163 -- SHOW GLOBAL VARIABLES LIKE 'local_infile';
```

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
32	12:41:49	ALTER TABLE transactions DROP FOREIGN KEY fk_card_id	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.063 sec
33	12:47:25	SELECT u.id, u.name, u.sname FROM users u WHERE u.id IN (SELECT t.user_id FROM transactions t GROUP BY t.us...	4 row(s) returned	0.157 sec / 0.000 sec
34	12:48:31	SELECT cc.id, cc.name, AVG(t.amount) AS media_amount FROM transactions t JOIN credit_cards cc ON t.card_id = cc.id JOIN com...	371 row(s) returned	0.032 sec / 0.000 sec
35	12:51:33	create table if not exists status_credit_cards (credit_card_id varchar(255) primary key, status_card enum ('active','inactive'), FOREI...	0 row(s) affected	0.141 sec

añadimos datos a la nuestra tabla

```
155 • INSERT INTO status_credit_cards (credit_card_id, status_card)
156 -- petition--
157 SELECT card_id,
158 CASE
159   WHEN SUM(declined) = 3 THEN 'inactive'
160   ELSE 'active'
161 END AS status
162 FROM (
163   SELECT
164     t.card_id,
165     t.declined,
166     ROW_NUMBER() OVER (PARTITION BY t.card_id ORDER BY t.timestamp DESC) AS rn
167   FROM transactions t
168 ) AS sub
169 WHERE rn <= 3
170 GROUP BY card_id;
171 • select count(*) AS card_activas
172 from status_credit_cards
173 where status_card = 'active';
174
175
176
177
178
```

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
36	12:53:13	select count(*) AS card_activas from status_credit_cards where status_card = 'active' LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
37	12:56:04	INSERT INTO status_credit_cards (credit_card_id, status_card) -- petition-- SELECT card_id, CASE WHEN SUM(decl...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.796 sec

Peticion final para ver el status de las tarjetas

The screenshot shows a SQL IDE interface with a query editor and a results pane. The query is as follows:

```
156 -- petition--
157 SELECT card_id,
158        CASE
159          WHEN SUM(declined) = 3 THEN 'inactive'
160          ELSE 'active'
161        END AS status
162 FROM (
163   SELECT
164     t.card_id,
165     t.declined,
166     ROW_NUMBER() OVER (PARTITION BY t.card_id ORDER BY t.timestamp DESC) AS rn
167   FROM transactions t
168 ) AS sub
169 WHERE rn <= 3
170 GROUP BY card_id;
171 select count(*) AS card_activas
172 from status_credit_cards
173 where status_card = 'active';
```

The results pane shows a single row in the 'card_activas' table with the value 4995.

card_activas
4995

Below the results, the 'Output' pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
37	12:56:04	INSERT INTO status_credit_cards (credit_card_id, status_card) -- petition-- SELECT card_id, CASE WHEN SUM(decl...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.796 sec
38	12:57:12	select count(*) AS card_activas from status_credit_cards where status_card = 'active' LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec

Crea una tabla con la que podamos unir los datos del nuevo archivo products.csv con la base de datos creada, teniendo en cuenta que desde transaction tienes product_ids. Genera la siguiente consulta:

Creamos la tabla products

```
47 company_name VARCHAR(255) ,
48 phone VARCHAR(255) ,
49 email VARCHAR(255) ,
50 country VARCHAR(255) ,
51 website VARCHAR (255)
52 );
53 CREATE TABLE IF NOT EXISTS products (
54 id VARCHAR (255) PRIMARY KEY ,
55 product_name VARCHAR(255) ,
56 price VARCHAR(255) ,
57 colour VARCHAR(255),
58 weight VARCHAR(255) ,
59 warehouse_id VARCHAR (255)
60 );
61 -- pasamos datos a nuestras tablas creadas
62 LOAD DATA LOCAL INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/transactions.csv'
63 INTO TABLE transactions
64 FIELDS terminated by ',';
```

Output

#	Time	Action	Message	Duration / Fetch
38	12:57:12	select count(*) AS card_activas from status_credit_cards where status_card = 'active' LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
39	13:01:07	CREATE TABLE IF NOT EXISTS products (id VARCHAR (255) PRIMARY KEY , product_name VARCHAR(255) , price V...	0 row(s) affected	0.109 sec

añadimos los datos a la tabla products

```
96
97 LOAD DATA LOCAL INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/products.csv'
98 INTO TABLE products
99 FIELDS terminated by ','
100 enclosed by '"'
101 ignore 1 rows
102 ;
103
104 -- creamos relaciones (FK) --
105
106 ALTER TABLE transactions
107 ADD CONSTRAINT fk_user_id
108 FOREIGN KEY (user_id) REFERENCES users(id);
109
110 ALTER TABLE transactions
111 ADD CONSTRAINT fk_card_id
112 FOREIGN KEY (card_id) REFERENCES credit_cards(id);
```

Output

#	Time	Action	Message	Duration / Fetch
39	13:01:07	CREATE TABLE IF NOT EXISTS products (id VARCHAR (255) PRIMARY KEY , product_name VARCHAR(255) , price V...	0 row(s) affected	0.109 sec
40	13:03:43	LOAD DATA LOCAL INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/products.csv' INTO TABLE products FIELDS ...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.016 sec

revisamos los datos añadidos

```

237 -- Necesitamos conocer el número de veces que se ha vendido cada producto.
238 • SELECT tp.product_id, p.product_name, count(*) AS num_veces_vendido
239 FROM transaction_products tp
240 JOIN products p
241 ON tp.product_id = p.id
242 JOIN transactions t
243 ON t.id = tp.transaction_id
244 WHERE t.declined = 0
245 GROUP By tp.product_id, p.product_name
246 ORDER BY num_veces_vendido DESC;
247
248 -- select*from transaction_products; -- revisamos si todo bien
249 -- select*from transactions;
250 • select*from products;
251
252
253
254

```

id	product_name	price	colour	weight	warehouse_id
1	Direwolf Stannis	\$161.11	#7c7c7c	1	WH-4
2	Tarly Stark	\$9.24	#919191	2	WH-3
3	duel tourney Lannister	\$171.13	#d8d8d8	1.5	WH-2
4	warden south duel	\$71.89	#111111	3	WH-1
5	skywalker ewok	\$171.22	#dbdbdb	3.2	WH-0
6	dooku solo	\$136.60	#c4c4c4	0.8	WH--1

products 33

Output

Action Output

#	Time	Action	Message	Duration / Fetch
95	20:53:12	SELECT tp product_id, p product_name, count(*) AS num_veces_vendido FROM transaction_products tp...	100 row(s) returned	1.453 sec / 0.000 sec
96	21:12:04	select*from products	100 row(s) returned	0.000 sec / 0.000 sec

creamos una tabla temporal "trans_prod_temp" que tenga la columna "products_ids_json" que tendrá los valores en el formato JSON
 -- para qual podamos extraer los valores de "product_ids" con
 JSON_TABLE()

```

176
177
178
179 -- Creamos una tabla temporal le llamamos "trans_prod_temp" que tenga la columna "products_ids_json" que tendrá los valores en el formato JSON
180 -- para qual podamos extraer los valores de "product_ids" con JSON_TABLE()
181
182 • CREATE TEMPORARY TABLE IF NOT EXISTS trans_prod_temp AS
183 SELECT id, product_ids, CONCAT('[',REPLACE( product_ids, " ", ""),'']') AS products_ids_json
184 FROM transactions;
185
186
187
188
189 -- drop temporary table trans_prod_temp;
190
191 -- Creamos la tabla nueva "transaction_products" con las columnas "transaction_id" y la "product_id"
192 • CREATE TABLE IF NOT EXISTS transaction_products AS
193 SELECT id AS transaction_id, product_id from

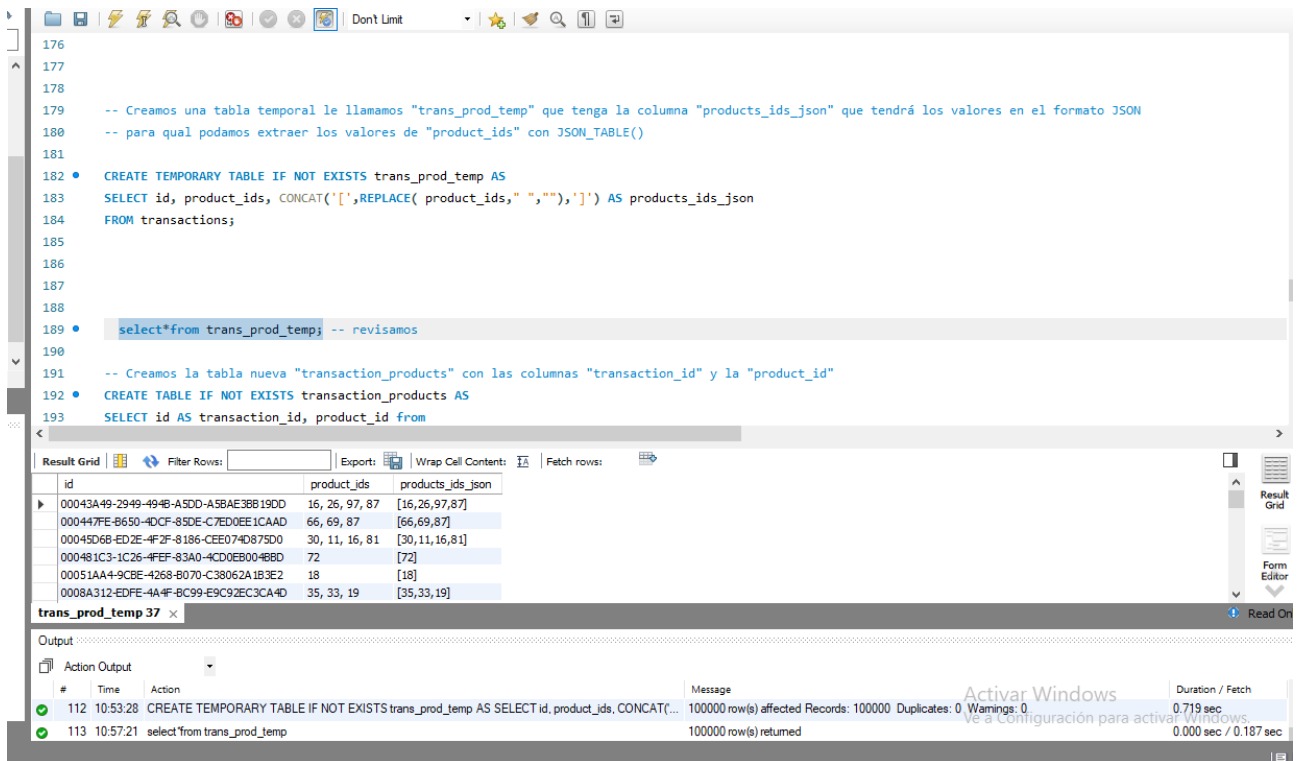
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
111	10:53:08	drop temporary table trans_prod_temp	0 row(s) affected	0.016 sec
112	10:53:28	CREATE TEMPORARY TABLE IF NOT EXISTS trans_prod_temp AS SELECT id, product_ids, CONCAT(...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	0.719 sec

revisamos



```
176
177
178
179 -- Creamos una tabla temporal le llamamos "trans_prod_temp" que tenga la columna "products_ids_json" que tendrá los valores en el formato JSON
180 -- para qual podamos extraer los valores de "product_ids" con JSON_TABLE()
181
182 • CREATE TEMPORARY TABLE IF NOT EXISTS trans_prod_temp AS
183 SELECT id, product_ids, CONCAT('[',REPLACE( product_ids," ",""),',]') AS products_ids_json
184 FROM transactions;
185
186
187
188
189 • select*from trans_prod_temp; -- revisamos
190
191 -- Creamos la tabla nueva "transaction_products" con las columnas "transaction_id" y la "product_id"
192 • CREATE TABLE IF NOT EXISTS transaction_products AS
193 SELECT id AS transaction_id, product_id from
```

id	product_ids	products_ids_json
00043A49-2949-494B-A5DD-A5BAE3B819DD	16, 26, 97, 87	[16,26,97,87]
000447FE-B650-4DCF-8SDE-C7ED0EE1CAAD	66, 69, 87	[66,69,87]
00045D68-ED2E-4F2F-8186-CEE074D875DD	30, 11, 16, 81	[30,11,16,81]
000481C3-1C26-4FEF-83A0-4CDOEB004BBD	72	[72]
00051AA4-9CBE-4268-B070-C38062A1B3E2	18	[18]
0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	35, 33, 19	[35,33,19]

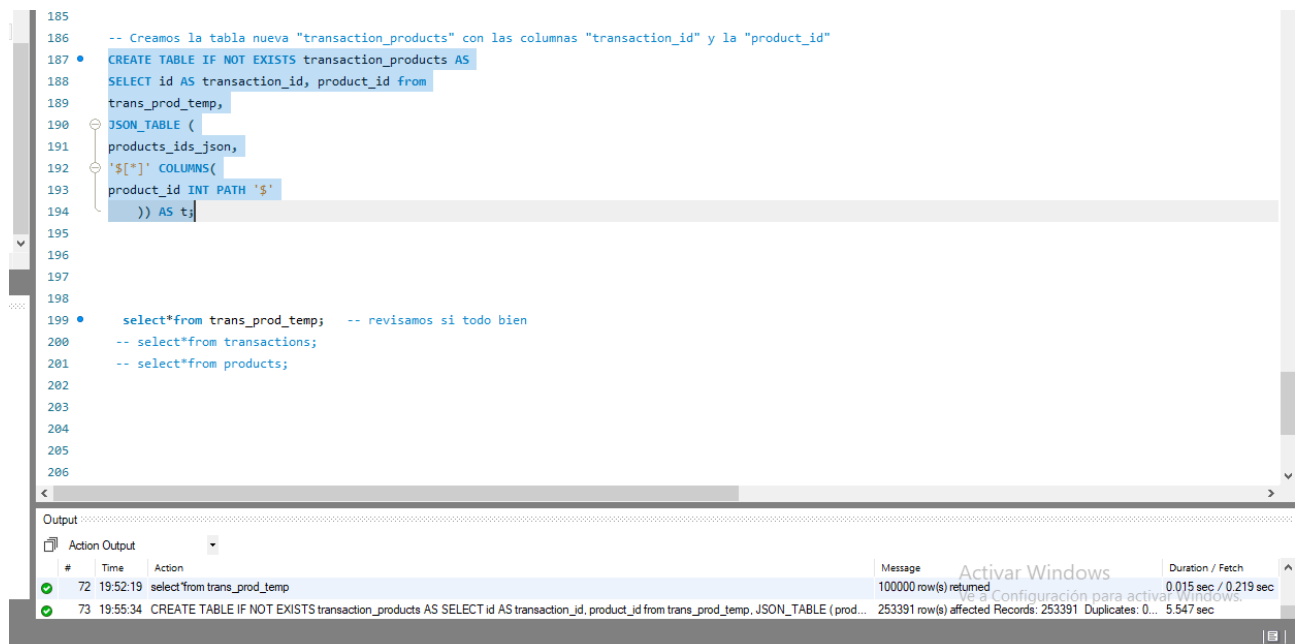
trans_prod_temp 37

Output

Action Output

#	Time	Action	Message	Duration / Fetch
112	10:53:28	CREATE TEMPORARY TABLE IF NOT EXISTS trans_prod_temp AS SELECT id, product_ids, CONCAT(...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	0.719 sec
113	10:57:21	select*from trans_prod_temp	100000 row(s) returned	0.000 sec / 0.187 sec

Creamos la tabla nueva "transaction_products" con las columnas "transaction_id" y la "product_id"



```
185
186 -- Creamos la tabla nueva "transaction_products" con las columnas "transaction_id" y la "product_id"
187 • CREATE TABLE IF NOT EXISTS transaction_products AS
188 SELECT id AS transaction_id, product_id from
189 trans_prod_temp,
190 JSON_TABLE (
191 products_ids_json,
192 '$[*]' COLUMNS(
193 product_id INT PATH '$'
194 )) AS t;
195
196
197
198
199 • select*from trans_prod_temp; -- revisamos si todo bien
200 -- select*from transactions;
201 -- select*from products;
202
203
204
205
206
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
72	19:52:19	select*from trans_prod_temp	100000 row(s) returned	0.015 sec / 0.219 sec
73	19:55:34	CREATE TABLE IF NOT EXISTS transaction_products AS SELECT id AS transaction_id, product_id from trans_prod_temp, JSON_TABLE (prod...	253391 row(s) affected Records: 253391 Duplicates: 0...	5.547 sec

revisamos

Query 1: Sprint_4_1* x Sprint_4* transactions_products

```
183 SELECT id, product_ids, CONCAT('[' , product_ids , ']') AS products_ids_json
184 FROM transactions;
185
186 -- Creamos la tabla nueva "transaction_products" con las columnas "transaction_id" y la "product_id"
187 CREATE TABLE IF NOT EXISTS transaction_products AS
188 SELECT id AS transaction_id, product_id from
189 trans_prod_temp,
190 JSON_TABLE (
191 products_ids_json,
192 '$[*]' COLUMNS(
193 product_id INT PATH '$'
194 )) AS t;
195
196
197
198
199 select*from transaction_products; -- revisamos si todo bien
200 -- select*from transactions;
```

Result Grid

transaction_id	product_id
00043A49-2949-494B-A5DD-ASBAE3BB19DD	16
00043A49-2949-494B-A5DD-ASBAE3BB19DD	26
00043A49-2949-494B-A5DD-ASBAE3BB19DD	97
00043A49-2949-494B-A5DD-ASBAE3BB19DD	87
000447FE-8650-4DCF-85DE-C7ED0EE1CAAD	66
000447FE-8650-4DCF-85DE-C7ED0EE1CAAD	69

transaction_products 26 x

Output

#	Time	Action	Message	Duration / Fetch
73	19:55:34	CREATE TABLE IF NOT EXISTS transaction_products AS SELECT id AS transaction_id, product_id from trans_prod_temp, JSON_TABLE (prod...	253391 row(s) affected Records: 253391 Duplicates: 0	5.547 sec
74	19:57:04	select*from transaction_products	253391 row(s) returned	0.000 sec / 0.375 sec

Conectamos con PRIMARY KEY de dos columnas la "transaction_id" y la "product_id"

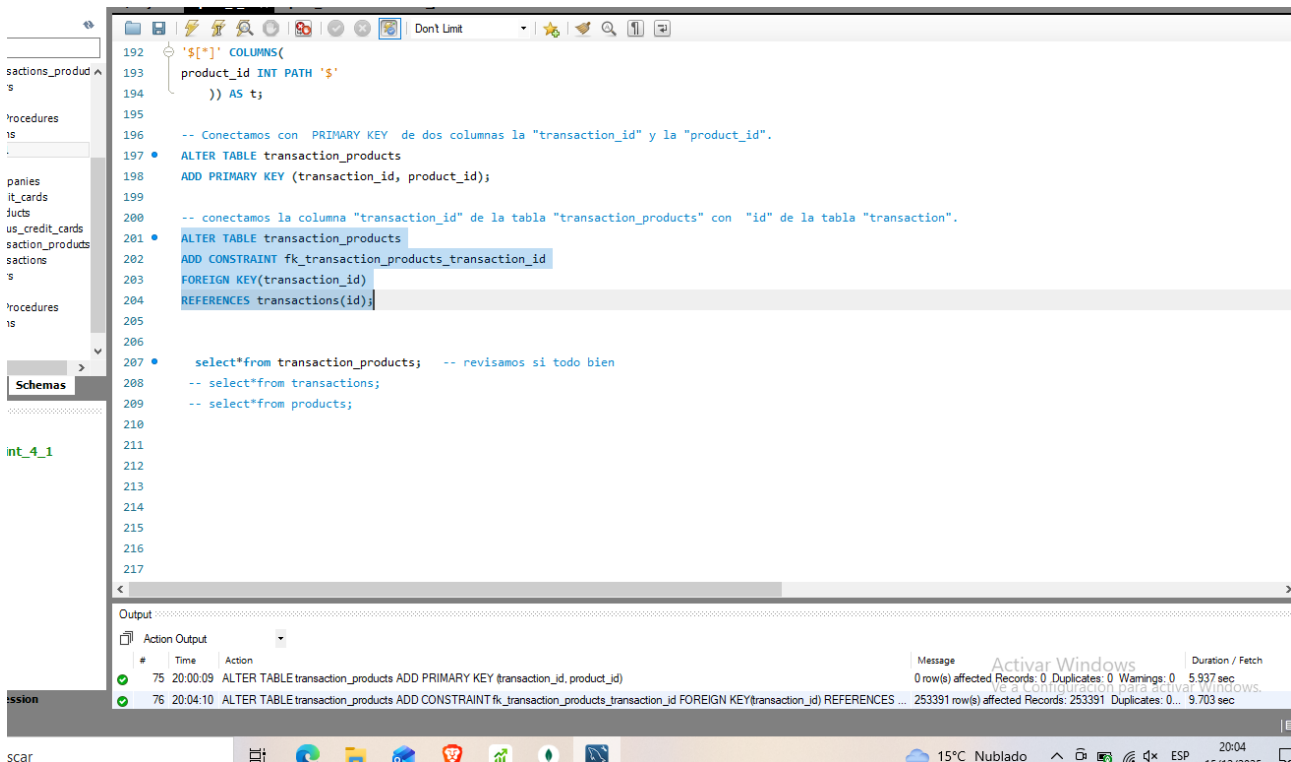
Query 1: Sprint_4_1* x Sprint_4* transactions_products

```
189 trans_prod_temp,
190 JSON_TABLE (
191 products_ids_json,
192 '$[*]' COLUMNS(
193 product_id INT PATH '$'
194 )) AS t;
195
196 -- Conectamos con PRIMARY KEY de dos columnas la "transaction_id" y la "product_id".
197 ALTER TABLE transaction_products
198 ADD PRIMARY KEY (transaction_id, product_id);
199
200
201 select*from transaction_products; -- revisamos si todo bien
202 -- select*from transactions;
203 -- select*from products;
```

Output

#	Time	Action	Message	Duration / Fetch
74	19:57:04	select*from transaction_products	253391 row(s) returned	0.000 sec / 0.375 sec
75	20:00:09	ALTER TABLE transaction_products ADD PRIMARY KEY (transaction_id, product_id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	5.937 sec

conectamos la columna "transaction_id" de la tabla "transaction_products" con "id" de la tabla "transaction"



```
192 '[$*]' COLUMNS(
193     product_id INT PATH '$'
194 ) AS t;
195
196 -- Conectamos con PRIMARY KEY de dos columnas la "transaction_id" y la "product_id".
197 • ALTER TABLE transaction_products
198     ADD PRIMARY KEY (transaction_id, product_id);
199
200 -- conectamos la columna "transaction_id" de la tabla "transaction_products" con "id" de la tabla "transaction".
201 • ALTER TABLE transaction_products
202     ADD CONSTRAINT fk_transaction_products_transaction_id
203     FOREIGN KEY(transaction_id)
204     REFERENCES transactions(id);
205
206
207 • select*from transaction_products; -- revisamos si todo bien
208 -- select*from transactions;
209 -- select*from products;
```

Output

#	Time	Action	Message	Duration / Fetch
75	20:00:09	ALTER TABLE transaction_products ADD PRIMARY KEY (transaction_id, product_id)	0 row(s) affected, Records: 0, Duplicates: 0, Warnings: 0	5.937 sec
76	20:04:10	ALTER TABLE transaction_products ADD CONSTRAINT fk_transaction_products_transaction_id FOREIGN KEY(transaction_id) REFERENCES transactions(id)	253391 row(s) affected, Records: 253391, Duplicates: 0, Warnings: 0	9.703 sec

conectamos la columna "product_id" de la tabla "transaction_products" con "id" de la tabla "products". Da error por los diferentes variables INT y VARCHAR

```
206 -- conectamos la columna "product_id" de la tabla "transaction_products" con "id" de la tabla "products".
207 • ALTER TABLE transaction_products
208     ADD CONSTRAINT fk_transaction_products_product_id
209     FOREIGN KEY(product_id)
210     REFERENCES products(id);
211
212 • select*from transaction_products; -- revisamos si todo bien
213 -- select*from transactions;
214 -- select*from products;
```

Output

#	Time	Action	Message	Duration / Fetch
77	20:08:29	ALTER TABLE products ADD PRIMARY KEY(id)	Error Code: 1068. Multiple primary key defined	0.016 sec
78	20:15:02	ALTER TABLE transaction_products ADD CONSTRAINT fk_transaction_products_product_id FOREIGN KEY(product_id) REFERENCES products(id)	Error Code: 3780. Referencing column 'product_id' and referenced column 'id' in foreign key constraint have different data types: int and varchar	0.015 sec

modificamos product_id_por_INT

```
211 -- modificamos product_id_por_INT
212 • ALTER TABLE transaction_products
213   MODIFY product_id INT;
214
215 • select*from transaction_products; -- revisamos si todo bien
216 -- select*from transactions;
217 -- select*from products;
218
219
220
221
222
223
```

Output

#	Time	Action	Message	Duration / Fetch
79	20:24:23	SELECT * FROM sprint_4_1.transaction_products	253391 row(s) returned	0.000 sec / 0.297 sec
80	20:24:55	ALTER TABLE transaction_products MODIFY product_id INT	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec

modificamos id_por_INT

```
219
220 -- modificamos id_por_INT
221 • ALTER TABLE products
222   MODIFY id INT NOT NULL;
223
224
225 • select*from transaction_products; -- revisamos si todo bien
226 -- select*from transactions;
227 -- select*from products;
```

Output

#	Time	Action	Message	Duration / Fetch
88	20:35:07	SHOW CREATE TABLE transaction_products	1 row(s) returned	0.015 sec / 0.000 sec
89	20:39:50	ALTER TABLE products MODIFY id INT NOT NULL	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0	0.235 sec

y ahora no nos da error

```
06 -- conectamos la columna "product_id" de la tabla "transaction_products" con "id" de la tabla "products".
07 • ALTER TABLE transaction_products
08   ADD CONSTRAINT fk_transaction_products_product_id
09   FOREIGN KEY(product_id)
10   REFERENCES products(id);
11
12 • SHOW CREATE TABLE products;
13
14
15 -- modificamos product_id_por_INT
16
17 • ALTER TABLE transaction_products
18   MODIFY product_id INT NOT NULL;
19
20 -- modificamos id_por_INT
21 • ALTER TABLE products
22   MODIFY id INT NOT NULL;
23
24
```

Output

#	Time	Action	Message	Duration / Fetch
89	20:39:50	ALTER TABLE products MODIFY id INT NOT NULL	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0	0.235 sec
90	20:40:55	ALTER TABLE transaction_products ADD CONSTRAINT fk_transaction_products_product_id FOREIGN ...	253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0	6.422 sec

revisamos FK creados

```
224 -- revisamos FOREIGN KEYS creados.
225 • SELECT
226     table_name,
227     column_name,
228     constraint_name,
229     referenced_table_name,
230     referenced_column_name
231 FROM
232     INFORMATION_SCHEMA.KEY_COLUMN_USAGE
233 WHERE
234     referenced_table_schema = 'transactions_sp4' AND table_name = 'transaction_products';
```

TABLE_NAME	COLUMN_NAME	CONSTRAINT_NAME	REFERENCED_TABLE_NAME	REFERENCED_COLUMN_NAME
------------	-------------	-----------------	-----------------------	------------------------

KEY_COLUMN_USAGE 31

Output

#	Time	Action	Message	Duration / Fetch
93	20:48:59	SELECT tp.product_id, p.product_name, count(*) AS num_veces_vendido FROM transaction_products tp...	100 row(s) returned	1.500 sec / 0.000 sec
94	20:50:59	SELECT table_name, column_name, constraint_name, referenced_table_name, referenced_...	0 row(s) returned	0.031 sec / 0.000 sec

Ejercicio 1

Necesitamos conocer el número de veces que se ha vendido cada producto.
Creamos la tabla products

```
236 -- Ejercicio_1
237 -- Necesitamos conocer el número de veces que se ha vendido cada producto.
238 • SELECT tp.product_id, p.product_name, count(*) AS num_veces_vendido
239 FROM transaction_products tp
240 JOIN products p
241 ON tp.product_id = p.id
242 JOIN transactions t
243 ON t.id = tp.transaction_id
244 WHERE t.declined = 0
245 GROUP BY tp.product_id, p.product_name
246 ORDER BY num_veces_vendido DESC;
```

product_id	product_name	num_veces_vendido
52	riverlands the duel	2642
29	Tully maester Tarly	2627
21	duel Direwolf	2603
16	the duel warden	2602
33	duel warden	2593
87	sith Jade	2591

Result 32

Output

#	Time	Action	Message	Duration / Fetch
94	20:50:59	SELECT table_name, column_name, constraint_name, referenced_table_name, referenced_...	0 row(s) returned	0.031 sec / 0.000 sec
95	20:53:12	SELECT tp.product_id, p.product_name, count(*) AS num_veces_vendido FROM transaction_products tp...	100 row(s) returned	1.453 sec / 0.000 sec

Diagrama final

